

p1054R0

A Unified Futures Proposal for C++

Draft Proposal, 7 May 2018

This version:

wg21.link/P1054

Authors:

[Lee Howes](#) (Facebook)

[Bryce Adelstein Leibach](#) (NVIDIA)

[David S. Hollman](#) (Sandia National Labs)

[Michał Dominiak](#) (Nokia)

Audience:

SG1

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Source:

github.com/executors/futures/blob/master/futures.bs

Table of Contents

1	Introduction
1.1	The Future/Promise Execution Model
1.2	Interactions with Executors
1.3	The Exception Handling Model
1.4	The Future Cancellation Model
1.5	<code>SemiFuture</code> and <code>.via</code> For Composition
1.6	Consuming Interfaces Are Rvalue Reference Qualified
1.7	Blocking Interfaces (<code>.get</code> and <code>.wait</code>)
1.8	Future Work
1.9	Prior Work
2	Proposed New Wording
2.1	The Future/Promise Execution Model
2.2	<code>FutureContinuation</code> Requirements
2.3	<code>Promise</code> Requirements
2.3.1	Promise Contract Executor Properties
2.4	<code>SemiFuture</code> Requirements
2.5	<code>ContinuableFuture</code> Requirements
2.6	<code>SharedFuture</code> Requirements
2.7	<code>std::execution::promise</code>

2.8	<code>std::execution::semi_future</code>
2.9	<code>std::execution::continuable_future</code>
2.10	<code>std::execution::shared_future</code>
2.11	<code>std::execution::make_promise_contract</code>
2.12	Generic Future Blocking Functions
2.13	<code>FutureContinuation</code> Helper Functions
2.14	Proposed Modifications to Executors
2.14.1	<code>Future</code> Requirements
2.14.2	<code>TwoWayExecutor</code> Requirements
2.14.3	<code>ThenExecutor</code> Requirements
2.14.4	<code>BulkTwoWayExecutor</code> Requirements
2.14.5	<code>BulkThenExecutor</code> Requirements
2.14.6	<code>twoway_t</code> Customization Points
2.14.7	<code>single_t</code> Customization Points
2.14.8	Properties To Indicate If Blocking And Directionality May Be Adapted
2.14.9	Class Template <code>executor</code>
2.14.10	<code>executor</code> Operations
2.14.11	<code>static_thread_pool</code> Executor Type

Index

Terms defined by this specification

References

Informative References

Contributors:

Marshall Cline	Carter Edwards	Jay Feldblum
Andrii Grynenco	Jared Hoberock	Hartmut Kaiser
Chris Kohlhoff	Chris Mysen	Eric Niebler
Sean Parent	Cory Perry	Felix Petriconi
Kirk Shoop	Mathias Stearn	

§ 1. Introduction

This paper introduces a hierarchy of concepts for future types that are designed to:

- *Interoperate With Executors*: The concepts should require the functionality needed by [executors](#).
- *Compose With Each Other*: The concepts should require the types to be composable.
- *Stay Skinny*: The concepts should require absolutely nothing else so that it is not burdensome to write future types.

There are five concepts introduces in this paper:

- `FutureContinuation`, invocable objects that are called with the value or exception of a future as an argument.

- `SemiFuture`, which can be bound to an executor, an operation which produces a `ContinuableFuture` (`f = sf.via(exec)`).
- `ContinuableFuture`, which refines `SemiFuture` and instances can have one `FutureContinuation` attached to them (`f.then(c)`), which is executed on the future's associated executor when the future becomes ready.
- `SharedFuture`, which refines `ContinuableFuture` and instances can have multiple `FutureContinuations` attached to them.
- `Promise`, each of which is associated with a future and make the future ready with either a value or an exception.

Or, described another way:

```

template <typename T>
struct FutureContinuation
{
    // At least one of these two overloads exists:
    auto operator()(T value);
    auto operator()(exception_arg_t, exception_ptr exception);
};

template <typename T>
struct SemiFuture
{
    template <typename Executor>
    ContinuableFuture<Executor, T> via(Executor&& exec) &&;
};

template <typename Executor, typename T>
struct ContinuableFuture
{
    template <typename RExecutor>
    ContinuableFuture<RExecutor, T> via(RExecutor&& exec) &&;

    template <typename Continuation>
    ContinuableFuture<Executor, auto> then(Continuation&& c) &&;
};

template <typename Executor, typename T>
struct SharedFuture
{
    template <typename RExecutor>
    ContinuableFuture<RExecutor, auto> via(RExecutor&& exec);

    template <typename Continuation>
    SharedFuture<Executor, auto> then(Continuation&& c);
};

template <typename T>
struct Promise
{
    void set_value(T value) &&;

    template <typename Error>
    void set_exception(Error exception) &&;
    bool valid() const;
};

```

In the following sections, we describe some of the key aspects of our proposed design.

§ 1.1. The Future/Promise Execution Model

In the Concurrency TS v1, it is unspecified where a `.then` continuation will be run. There are a number of possible answers:

- Consumer Side: The consumer execution agent always executes the continuation. `.then` blocks until the producer execution agent signals readiness.
- Producer Side: The producer execution agent always executes the continuation. `.set_value` blocks until the consumer execution agent signals readiness.
- `inline_executor` Semantics: If the shared state is ready when the continuation is set, the consumer thread executes the continuation. If the shared state is not ready when the continuation is set, the producer thread executes the continuation.
- `thread_executor` Semantics: A new `std::thread` executes the continuation.

This is a source of trouble ([P0701r0] and [P0701r1]). The first two answers are undesirable, as they would require blocking, which is not ideal for an asynchronous interface. The third and fourth are likewise distasteful, as they can be vague or inefficient (respectively).

Executors, finally, give us at least a partial solution to this problem. The question changes to "where do we enqueue work into the executor"? The answer: work is always enqueued on the consumer side as if, but not necessarily, via `then_execute`. You can query executor properties to determine whether or not the executor's APIs will block, which tells you whether or not continuation `attachment` (consumer side) or future `fulfillment` (producer side) potentially blocks pending execution (e.g. `inline_executor` semantics).

§ 1.2. Interactions with Executors

The executors proposal defines a collection of executor types intended for use managing the execution of tasks on resources. There are three fundamental executor categories that cover directionality and control of launch:

- one-way
- two-way
- then

The first two could be considered *immediately launched*. That is that once handed to the executor, they may start immediately, assuming the internal executor policies and resources allow it. This makes them very useful for lazy-launch scenarios.

Lazy launch scenarios are common in callback-based code, and in a wide range of future library implementations such as HPX and folly. In these designs, a callback is executed on completion of some asynchronous work, and that callback enqueues work into the executor. This means that work is enqueued only after all dependencies are satisfied.

Then-executors, on the other hand, are intended for explicitly deferred work. Work can be handed to the executor dependent on prior work, before that prior work is completed. This design is fundamentally different, but offers scope for optimization by the executor of chains of dependencies that it can batch, without running additional code on completion of each.

The current executor design is intentionally generic - it makes few requirements on the future types it can use as input dependencies for the `then_execute` and `bulk_then_execute` operations. We can assume that for a future returned by a previous call to `then_execute` or `bulk_then_execute`, the executor understands the implementation of the future can perform whatever dependence tracking and optimization necessary. This then is an implementation detail.

However, there will also be interactions where a task to run on one executor is dependent on one produced by another. For this to be practical, we need a standardised mechanism to tie the two executors together. This amounts to a standard API for triggering deferred work.

To solve this we provide two things:

- A promise concept, that allows setting of value and/or exception.
- A mechanism to retrieve from an executor a pair of a promise and a future, such that the future is a valid input dependence for a call to `then_execute` or `bulk_then_execute`.

The promise is a write-only concept. This simplifies the definition and improves flexibility.

The future is not a full future in the sense of future concepts. It is merely a token that completes when the promise is satisfied. This means that it is useful only for connecting to `then_execute` or `bulk_then_execute` on the executor that provided the result.

§ 1.3. The Exception Handling Model

Our proposal moves away from future continuations that take futures as an argument, the design used in the Concurrency TS v1. Instead, continuations take the value type of the future. This, however, requires a new exception handling model. Both the executors proposal and this paper adopt the same model, where users can provide callable objects with both a value type and an optional tag disambiguated exception handling overload.

In the example below, let `f` and `g` be `ContinuableFutures` and let `c` be a `FutureContinuation`.

```
g = f.then(c);
```

When `f` is fulfilled:

- If `f` has a value `val`:
- If `c(val)` is well-formed, it is called and the future `g` is fulfilled with the result or any exception thrown from the evaluation.
- Otherwise, the future `g` is fulfilled with the value `val` (this is known as *future value propagation* - see [\[P0701r0\]](#) and [\[P0701r1\]](#)).
- If `f` fails with exception `exc`:
- If `c(exception_arg, exc)` is well-formed, it is called and the future `g` is fulfilled with the result or any exception thrown from the evaluation.
- Otherwise, the future `g` is fulfilled with the exception `exc` (this is known as *future exception propagation* - see [\[P0701r0\]](#) and [\[P0701r1\]](#)).

Note that if both overloads are defined, `decltype(G(exception_arg, err))` shall be convertible to `decltype(G(val))`.

This paper defines some helper types which can be used to build callable arguments that meet the `FutureContinuation` arguments. For example, `on_value_or_error(f, g)` takes two callables (a on-value continuation and on-error continuation) and returns a single `FutureContinuation` object composed from the two.

§ 1.4. The Future Cancellation Model

This paper proposes a way to hook a [cancellation notifier](#) into a future/promise pair, using [cancellable_promise_contract_t](#). It allows for authors of future types to support cancellation if they want to, and to correctly hook into the cancellation mechanisms of other futures they interoperate with - such as inside [.via](#), where the implementation can create a future/promise pair that will, on cancellation, invoke a function that will cancel the current future.

A sample implementation of [.via](#) could be as follows:

```
template <typename Executor>
auto via(Executor && ex) &&
{
    auto cancel = [this] { handle_cancellation(); };
    auto [promise, future] = execution::query(ex, cancellable_promise_contract_t{ cancel });
    then([promise = std::move(promise)](auto value) { promise.set_value(std::move(value)); });
    return future;
}
```

We are currently not proposing a concrete cancellation API; we plan to do that in a future revision of the paper. At this time, there is no guarantee that a future honors a [cancellation notifier](#), and no requirements as to when it actually invokes it.

§ 1.5. [SemiFuture](#) and [.via](#) For Composition

The [SemiFuture](#) concept and [.via](#) mechanism ([P0783r0] and [P0904r0]) gives us control of the transfer of execution ownership between executors and a way to convert between different future types.

One problem that arises in the [executor model](#) is how a [future](#)-returning interface can dictate the executor that callers attach continuations to.

[SemiFuture](#) and [.via](#) is mechanism that allows the caller to control the executor that subsequent chaining operations use. An interface may return a handle to its work, a future in the most abstract sense, that does not provide a means to chain more work. This future will complete on whatever executor that interface was using. This returned future satisfies the [SemiFuture](#) concept, and the caller is hence aware that to make use of it they must attach an executor, using [.via](#), to transition from the interface's executor to the caller's executor. From that point on, the caller can use the future as necessary, safely enqueueing work onto a known executor, and protecting the interface from misuse.

Additionally, since [ContinuableFuture](#) and [SharedFuture](#)s are also [SemiFutures](#), [.via](#) provides a way to convert (if possible) from one future type (associated with a particular executor type) to another "foreign" future type (associated with another executor type).

As a simple example:

```

std::execution::semi_future<DataType> async_api() {
    std::execution::continuable_future<APIExecutor, SerializedDataType> =
        doAsyncWork();
    return std::move(f).then(
        [](SerializedDataType&& val){ return deserialize(val); });
}

void caller() {
    LocalExecutor e;
    auto sf = async_api();
    auto f = std::move(sf).via(e);
    std::move(f).then([](DataType&& data) {
        std::cout << "Name: " << data.name() << "\n";
    });
}

```

There is a strict separation of control here between `caller` and `async_api`.

§ 1.6. Consuming Interfaces Are Rvalue Reference Qualified

In a change to the model used in `std::future`, where `std::future::get()` is a consuming operation but is l-value qualified, consuming operations in this proposal are r-value qualified.

For free functions this should be obvious. If we had a free function `future_then` that takes a future and returns a value, we will likely r-value qualify it:

```

std::future<T2> future_then(std::future<T>&& f, continuation&& c);

```

For consistent and safe use, the same should apply to the equivalent builtin methods.

In chaining code, this falls out cleanly:

```

auto f2 = do_async_thing().then([](T val){return val;}).then([](T2 val){return val;});

```

Occasionally we must be explicit:

```

auto f = do_async_thing();
auto f2 = std::move(f).then([](T val){return val;}).then([](T2 val){return val;});

```

but this is a minor inconvenience given that it allows the tooling to warn on use-after-move and related misuses.

§ 1.7. Blocking Interfaces (`.get` and `.wait`)

The future concepts in this paper are intended to express the minimal requirements for executors interoperability and composition. Executors do not require a blocking interface, and futures do not require a blocking interface to compose. This paper proposes the addition of the free functions `std::this_thread::future_wait` and

`std::this_thread::future_get`. These functions will block on any `SemiFuture` type, and are suitable for use within `std::thread` execution agents. Some individuals may feel that blocking interfaces are more fundamental for futures than continuation chaining interfaces - some may desire to not provide continuation chaining interfaces at all. We believe that this is a perfectly valid design; however, such non-continuable futures are outside of the scope of this work.

- If you want to write a future that only has blocking interfaces and doesn't support continuations, that's fine! But that type of future cannot interoperate with executors.
- If you want to write a future that only doesn't have blocking interfaces and does support continuations, that's fine! That type of future can interoperate with executors, so you should conform to the proposed concepts.
- If you want to write a future that has blocking interfaces and supports continuations, that's fine! That type of future can interoperate with executors, so you should conform to the proposed concepts.

[\[P0701r0\]](#) and [\[P0701r1\]](#) discuss some of the challenges relating to future/promise synchronization that motivated the current design.

§ 1.8. Future Work

There is plenty of future work on futures to be done. Here is a list of topics that are currently on our agenda:

- Forward progress guarantees for futures and promises.
- Requirements on synchronization for use of futures and promises from non-concurrent execution agents.
- `std::future`/`std::promise` interoperability.
- Future unwrapping, both `future<future<T>>` and more advanced forms.
- `when_all`/`when_any`/`when_n`.
- `async`.

§ 1.9. Prior Work

This work is unification of prior papers written individually by the authors:

- [\[P0701r0\]](#)
- [\[P0701r1\]](#)
- [\[P0783r0\]](#)
- [\[P0904r0\]](#)

§ 2. Proposed New Wording

The following wording is purely additive to [the executors proposal](#).

§ 2.1. The Future/Promise Execution Model

1. A **future** is an object that executes, on an executor, **continuations** that are passed an [asynchronous result](#) as a parameter.

- A **uniquely owned future** moves from the [asynchronous result](#) and can have at most one [continuation](#) attached.
 - A **non-uniquely owned future** copies from the [asynchronous result](#) and can have one or more [continuations](#) attached.
2. A **promise** is an object that produces an [asynchronous result](#) and is associated with a [future](#). [*Note: Although a [promise](#) is associated with one [future](#), multiple [shared future](#) objects may refer to the same underlying [shared future](#). - end note*]
 3. The **status** of a [future](#):
 - Is either **ready**, **not ready**, or **invalid**.
 - Shall be [ready](#) if and only if the [future](#) holds a value or an exception ready for retrieval.
 4. An **asynchronous result** is either a value or an exception. The result is created by the [promise](#) and accessed by the [future](#).
 5. A [future](#) may optionally have a **cancellation notifier**, which is a nullary [callable object](#).
 6. When a [future](#) is **cancelled**:
 - If the [future](#) has a [cancellation notifier](#), it is called in the calling thread.
 - Then, the [future's](#) status is set to [invalid](#).
 7. When a [continuation](#) is **attached** to a [future](#):
 - First, the [status](#) of its [future](#) is checked.
 - Then,
 - If the [status](#) is [ready](#) or [not ready](#):
 - The [continuation](#) is enqueued for execution pending readiness of [future](#) on the [future's](#) executor, with the [asynchronous result](#) as its parameter.
 - Finally, if the [future](#) is a [uniquely owned future](#), the status is set to [invalid](#).
 - If the [status](#) is [invalid](#), the behavior is unspecified.
 8. When a [promise](#) **fulfills** its associated [future](#) with a value or error, if the lifetime of the [future](#) has not ended:
 - First, the [status](#) of its [future](#) is checked.
 - Then,
 - If the [status](#) is [not ready](#):
 - The [status](#) is set to [ready](#)
 - Finally, any execution agents that are blocking until the [status](#) is [ready](#) are unblocked.
 - If the [status](#) is [ready](#) or [invalid](#), the behavior is unspecified.

If the lifetime of the [future](#) has ended, [fulfillment](#) has no effect.
 9. [Fulfillment](#) of a [future](#) [synchronizes with](#) execution of [continuations](#) that were enqueued for execution by that [future](#).
 10. The lifetime of the [continuations](#) and the [asynchronous result](#) passed to the [continuations](#) shall last until execution of the [continuations](#) completes. [*Note: The [future](#) may move (if it is a [uniquely owned future](#)) or copy (if it is a [non-uniquely owned future](#)) from the result into a new object to ensure this behavior. - End Note*] [*Note: The [future](#) may move from [continuations](#) into new object to ensure this behavior. - End Note*]

11. The lifetime of the [continuations](#) and the [asynchronous result](#) passed to the [continuations](#) shall last until execution of the [continuations](#) completes. [*Note*: The [promise](#) may move or copy (if the result is [CopyConstructible](#)) from the result into a new object to ensure this behavior. - *End Note*] [*Note*: The [promise](#) may move from continuations into new object to ensure this behavior. - *End Note*]
12. Upon destruction of a [promise](#):
 - First, the [promise](#) checks the [status](#) of its [future](#).
 - Then, if the [status](#) is [not ready](#), the [promise](#) shall be [fulfilled](#).
13. Setting the [status](#) of a [future synchronizes with](#) operations that check the [future](#)'s status.
14. Operations that modify the set of continuations stored in a [future synchronize with](#) each other.
15. Successful [fulfillment](#) of a [future synchronizes with attachment](#) of [continuations](#) to that future.
16. Successful [attachment](#) of a [continuation](#) to a [future synchronizes with fulfillment](#) of the promise associated with that [future](#).
17. If a [future](#) has a [cancellation notifier](#), successful [fulfillment](#) of it's associated [promise synchronizes with cancellation](#) of the [future](#).

§ 2.2. [FutureContinuation](#) Requirements

```
namespace std::execution {

struct exception_arg_t { explicit exception_arg_t() = default; };

inline constexpr exception_arg_t exception_arg{};

template <typename F, typename T>
    struct is_future_value_continuation;

template <typename F, typename T>
    inline constexpr bool is_future_value_continuation_v
        = is_future_value_continuation<F, T>::value;

template <typename F>
    struct is_future_exception_continuation;

template <typename F>
    inline constexpr bool is_future_exception_continuation_v
        = is_future_exception_continuation<F>::value;

template <typename F, typename T>
    struct is_future_continuation;

template <typename F, typename T>
    inline constexpr bool is_future_continuation_v
        = is_future_continuation<F, T>::value;

}
```

1. A **future continuation** is a [callable object](#) that consumes the value of a [future](#).
2. The struct `exception_arg_t` is an empty structure type used as a unique type to disambiguate `FutureContinuation` overloads that are called when a [future](#) holds an exception from `FutureContinuation` overloads that are called when a [future](#) holds a value.
3. This sub-clause contains templates that may be used to determine at compile time whether a type meets the requirements of [future continuations](#) for a particular [future](#) value type. Each of these templates is a `BinaryTypeTrait` with a base characteristic of `true_type` if the corresponding condition is true, and `false_type` otherwise.
4. A `FutureContinuation` type shall meet the `MoveConstructible` requirements and the requirements described in the Tables below.

Type Property Queries

Template	Condition	Preconditions
<pre>template <typename F, typename T> struct is_future_value_continuation;</pre>	<pre>is_move_constructible_v<F> && is_invocable_v<F, T></pre>	<code>T</code> is a complete type.
<pre>template <typename F> struct is_future_exception_continuation;</pre>	<pre>is_move_constructible_v<F> && is_invocable_v<F, exception_arg_t, exception_ptr></pre>	<code>T</code> is a complete type.
<pre>template <typename F, typename T> struct is_future_continuation;</pre>	<pre>is_future_value_continuation_v<F, T> is_future_exception_continuation_v<F></pre>	<code>T</code> is a complete type.

§ 2.3. Promise Requirements

#. A `Promise` type for value type `T` and error type `E` shall meet the `MoveConstructible` requirements, the `MoveAssignable` requirements, and the requirements described in the Tables below.

Descriptive Variable Definitions

Variable	Definition
<code>T</code>	Either: • Any (possibly cv-qualified) object type that is not an array, or • (possibly cv-qualified) <code>void</code>.
<code>t</code>	a value of a type contextually convertible to <code>T</code>
<code>e</code>	a value of type contextually convertible to <code>exception_ptr</code>
<code>P<T></code>	A promise type for value type <code>T</code>
<code>p</code>	An rvalue of type <code>P<T></code>

Expression	Return Type	Operational semantics
<code>promise_value_t<P<T>></code>	<code>T</code>	

<code>p.set_value(t)</code>	<code>void</code>	• <i>Requires:</i> <code>!is_void_v<T></code> • <i>Effects:</i> Completes the promise and associated future with <code>t</code>.
<code>p.set_value()</code>	<code>void</code>	• <i>Requires:</i> <code>is_void_v<T></code> • <i>Effects:</i> Completes the promise and associated future.
<code>p.set_exception(e)</code>	<code>void</code>	Completes the promise and associated future with the error <code>e</code> .
<code>p.valid()</code>	Contextually convertible to <code>bool</code> .	<code>true</code> if the promise has an associated future that is incomplete, <code>false</code> otherwise.

§ 2.3.1. Promise Contract Executor Properties

```
template <class T>
struct promise_contract_t
{
    static constexpr bool is_requirable = false;
    static constexpr bool is_preferable = false;

    using polymorphic_query_result_type
        = std::pair<std::execution::promise<T>, std::execution::semi_future<T>>;
};

template <typename T>
inline constexpr promise_contract = promise_contract_t<T>{};
```

The `promise_contract_t` property can be used only with query.

The result of a query of the `promise_contract_t` property applied to a `ThenExecutor` or `BulkThenExecutor` is a `std::pair` consisting of a `Promise` and an implementation-defined token type that will be interpreted as a valid input future by calls to `then_execute` or `bulk_then_execute` and that is satisfied by calling `set_value` or `set_exception` on the promise.

The value returned from `execution::query(e, promise_contract_t<T>)`, where `e` is an executor and `T` is a type, should be unique for any given call. When `e` is a `ThenExecutor` or `BulkThenExecutor` the result of the query is a `std::pair` where first value is an instance of a type matching the `Promise` requirements and the second is a token type that `e` will interpret as a valid future parameter to calls to `then_execute` or `bulk_then_execute`.

```

template <T, C>
struct cancellable_promise_contract_t
{
    static constexpr bool is_requirable = false;
    static constexpr bool is_preferable = false;

    using polymorphic_query_result_type
        = std::pair<std::execution::promise<T>, std::execution::semi_future<T>>;

    template<class Executor>
    static constexpr decltype(auto) static_query_v
        = Executor::query(promise_contract_t());

    template<class Executor>
    friend std::pair<std::execution::promise<T>, std::execution::semi_future<T>>
        query(const Executor& ex, const cancellable_promise_contract_t&);

    CancellationNotifier cancellation_notifier;
};

```

The `cancellable_promise_contract_t` property can be used only with query.

`cancellable_promise_contract_t` differs from `promise_contract_t` in that the query carries a cancellation callback, `cancellation_notifier`, that will be called as `std::invoke(cancellation_notifier)` by the `ThenExecutor` on cancellation of the task dependent on the future resulting from the `cancellable_promise_contract_t` query.

The result of a query of the `cancellable_promise_contract_t` property applied to a `ThenExecutor` or `BulkThenExecutor` is a `std::pair` consisting of a `Promise` and an implementation-defined token type that will be interpreted as a valid input future by calls to `then_execute` or `bulk_then_execute`, that is satisfied by calling `set_value` or `set_exception` on the promise and that supports cancellation by the executor.

The value returned from `execution::query(e, cancellable_promise_contract_t<T>{cancellation_notifier})`, where `e` is an executor and `T` is a type, should be unique for any given call. When `e` is a `ThenExecutor` or `BulkThenExecutor` the result of the query is a `std::pair` where first value is an instance of a type matching the `Promise` requirements and the second is a token type that `e` will interpret as a valid future parameter to calls to `then_execute` or `bulk_then_execute`.

```

template<class Executor>
friend std::pair<std::execution::promise<T>, std::execution::semi_future<T>>
    query(const Executor& ex, const cancellable_promise_contract_t&);

```

- *Returns:* A `std::pair` where the first value is a `std::execution::promise<T>` and the second is a `std::execution::semi_future<T>` such that the future was retrieved from the promise.
- *Remarks:* This function shall not participate in overload resolution unless `executor_future_t<Executor, T>` is `std::execution::semi_future<T>`.

§ 2.4. `SemiFuture` Requirements

1. A ***semi future*** is an object that can be bound to an executor to produce a [future](#).
2. A `SemiFuture` type for type `T` shall meet the `MoveConstructible` requirements, the `MoveAssignable` requirements, and the requirements described in the Tables below.

Descriptive Variable Definitions

Variable	Definition
<code>T</code>	Either: • Any (possibly cv-qualified) object type that is not an array, or • (possibly cv-qualified) <code>void</code>.
<code>SF<T></code>	A <code>SemiFuture</code> type for value type <code>T</code> .
<code>sf</code>	An rvalue of type <code>SF<T></code> .
<code>E</code>	An executor type, either: • A <code>ThenExecutor</code> such that <code>execution::can_query_v<E, promise_contract_t<T>> == true</code>, or • A <code>OneWayExecutor</code> otherwise.
<code>e</code>	A value of type <code>E</code> .
<code>CF<T, E></code>	A <code>ContinuableFuture</code> type for value type <code>T</code> and executor type <code>E</code> , either: • <code>decltype(execution::query(e, promise_contract_t<T>()).second)</code> if <code>E</code> is a <code>ThenExecutor</code> and <code>execution::can_query_v<E, promise_contract_t<T>> == true</code>, or • <code>continuable_future<T, E></code> otherwise.

`SemiFuture` Requirements

Expression	Return Type	Operational Semantics
<code>future_value_t<SF<T>></code>	<code>T</code>	
<code>future_exception_t<SF<T>></code>	Implicitly convertible to <code>exception_ptr</code> .	
<code>sf.via(e)</code>	<i>implementation-defined</i>	Returns: A <code>ContinuableFuture</code> for value type <code>T</code> that is bound to the executor <code>e</code> and will be made ready with the value or exception of <code>sf</code> when <code>sf</code> is made ready. Throws: If <code>!execution::query(e, promise_contract_t<T>()).second</code>, throws any exception thrown by <code>!execution::query(e, oneway_t)</code>.

§ 2.5. `ContinuableFuture` Requirements

1. A ***continuable future*** is a [future](#) that is bound to an executor and can have continuations attached to it.

2. A `ContinuableFuture` shall meet the `SemiFuture` requirements and the requirements described in the Tables below.

Descriptive Variable Definitions

Variable	Definition
<code>E</code>	An executor type.
<code>e</code>	A value of type <code>E</code> .
<code>T</code>	Either: - Any (possibly cv-qualified) object type that is not an array, or (possibly cv-qualified) <code>void</code>.
<code>CF<T, E></code>	A <code>ContinuableFuture</code> type for value type <code>T</code> and executor type <code>E</code> .
<code>cf</code>	A value of type <code>CF<T, E></code> .
<code>rcf</code>	An rvalue of type <code>CF<T, E></code> .
<code>val</code>	The value contained within the successfully completed future <code>rcf</code> .
<code>ex</code>	The exception contained within the exceptionally completed future <code>rcf</code> .
<code>G</code>	Any type such that <code>is_future_continuation_v<G, T> == true</code> .
<code>g</code>	An object of type <code>G</code> .
<code>R</code>	Either: - If <code>is_same_v<remove_cv_t<T>, void> == true</code> and <code>invoke(g)</code> is well formed, <code>decltype(g())</code>. - If <code>is_same_v<remove_cv_t<T>, void> != true</code> and <code>invoke(g, val)</code> is well formed, <code>decltype(g(val))</code>. - Otherwise, <code>T</code>.
<code>SF<T></code>	A <code>SemiFuture</code> type for value type <code>T</code> .
<code>NORMAL</code>	Either: - If <code>is_same_v<remove_cv_t<T>, void> == true</code>, <code>DECAY_COPY(std::forward<G>(g))()</code>, and - Otherwise, <code>DECAY_COPY(std::forward<G>(g))(std::move(val))</code>.
<code>EXCEPTIONAL</code>	The expression <code>DECAY_COPY(std::forward<G>(g))(exception_arg, std::move(ex))</code> .

ContinuableFuture Requirements

Expression	Return Type	Operational Semantics
<code>cf.get_executor()</code>	<code>E</code>	Returns: The executor that the future is bound to.
<code>rcf.then(g)</code>	<code>CF<E, R></code>	Returns: A <code>ContinuableFuture</code> that is bound to the executor <code>e</code> and that wraps the type returned by execution of either the value or exception operations implemented in the continuation. Effects: When <code>rcf</code> becomes nonexceptionally ready, and if <code>NORMAL</code> is a well-formed expression, creates an execution agent which invokes <code>NORMAL</code> at most

once, with the call to `DECAY_COPY` being evaluated in the thread that called `.then`.

Otherwise, when `rcf` becomes exceptionally ready, if `EXCEPTIONAL` is a well-formed expression, creates an execution agent which invokes `EXCEPTIONAL` at most once, with the call to `DECAY_COPY` being evaluated in the thread that called `.then`.

If `NORMAL` and `EXCEPTIONAL` are both well-formed expressions, `decltype(EXCEPTIONAL)` shall be convertible to `R`.

If `NORMAL` is not a well-formed expression and `EXCEPTIONAL` is a well-formed expression, `decltype(EXCEPTIONAL)` shall be convertible to `decltype(val)`.

If neither `NORMAL` nor `EXCEPTIONAL` are well-formed expressions, the invocation of `.then` shall be ill-formed.

May block pending completion of `NORMAL` or `EXCEPTIONAL`.

The invocation of `.then` synchronizes with (C++Std [intro.multithread]) the invocation of `g`.

Fulfills the `ContinuableFuture` with the result of the `NORMAL` or `EXCEPTIONAL` expression, or any exception thrown by either. Otherwise, fulfills the `ContinuableFuture` with either `val` or `e`.

Synchronization: The destruction of the continuation that generates `rcf`'s value synchronizes with the invocation of `g` and with the destruction of `g`.

§ 2.6. SharedFuture Requirements

1. A **shared future** is a non-uniquely owned future that is copyable, is bound to an executor and that allows one or more continuation to be attached to it.
2. A `SharedFuture` shall meet the `ContinuableFuture` requirements, the `CopyConstructible` requirements, the `CopyAssignable` requirements and the requirements described in the Tables below.

Descriptive Variable Definitions

Variable	Definition
<code>E</code>	An executor type.
<code>e</code>	A value of type <code>E</code> .
<code>T</code>	Any (possibly cv-qualified) object type that is not an array.
<code>CF<T, E></code>	A <code>ContinuableFuture</code> type for executor type <code>E</code> and value type <code>T</code> .
<code>SHF<E, T></code>	A <code>SharedFuture</code> type for executor type <code>E</code> and value type <code>T</code> .
<code>shf</code>	A value of type <code>SHF<E, T></code> .
<code>NORMAL</code>	The expression <code>DECAY_COPY(std::forward<G>(g))(val)</code> if <code>T</code> is non-void and <code>DECAY_COPY(std::forward<G>(g))()</code> if <code>T</code> is void.
<code>EXCEPTIONAL</code>	The expression <code>DECAY_COPY(std::forward<G>(g))(exception_arg, ex)</code> ,

SharedFuture Requirements

Expression	Return Type	Operational Semantics
<code>shf.then(g)</code>	If <code>T</code> is non-void and <code>INVOKE(declval<G>(), declval<T>())</code> or if <code>T</code> is <code>void</code> and <code>INVOKE(declval<G>())</code> is well-formed: <code>CF<E, decltype(INVOKE(declval<G>(), declval<T>()))></code> Otherwise: <code>CF<T, E></code>	Returns: A <code>ContinuableFuture</code> that is bound to the executor <code>e</code> and that wraps the type returned by execution of either the value or exception operations implemented in the continuation. Effects: When <code>shf</code> becomes nonexceptionally ready, and if <code>NORMAL</code> is a well-formed expression, creates an execution agent which invokes <code>NORMAL</code> at most once, with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>.then</code>. Otherwise, when <code>shf</code> becomes exceptionally ready, if <code>EXCEPTIONAL</code> is a well-formed expression, creates an execution agent which invokes <code>EXCEPTIONAL</code> at most once, with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>.then</code>. If <code>NORMAL</code> and <code>EXCEPTIONAL</code> are both well-formed expressions, <code>decltype(EXCEPTIONAL)</code> shall be convertible to <code>R</code>. If <code>NORMAL</code> is not a well-formed expression and <code>EXCEPTIONAL</code> is a well-formed expression, <code>decltype(EXCEPTIONAL)</code> shall be convertible to <code>decltype(val)</code>. If neither <code>NORMAL</code> nor <code>EXCEPTIONAL</code> are well-formed expressions, the invocation of <code>.then</code> shall be ill-formed. May block pending completion of <code>NORMAL</code> or <code>EXCEPTIONAL</code>. The invocation of <code>.then</code> synchronizes with (C++Std [intro.multithread]) the invocation of <code>g</code>. <u>Fulfills</u> the <code>ContinuableFuture</code> with the result of the <code>NORMAL</code> or <code>EXCEPTIONAL</code> expression, or any exception thrown by either. Otherwise, <u>fulfills</u> the <code>ContinuableFuture</code> with either <code>val</code> or <code>e</code>. Postconditions: Has no observable affect on <code>sfh</code>.
<code>shf.via(e)</code>	Implementation-defined	Returns: A <code>ContinuableFuture</code> for type <code>T</code> that is bound to the executor <code>e</code>. Effect: Returns an implementation-defined <code>ContinuableFuture</code> onto which continuations can be attached that will run on <code>e</code>.

Success: Succeeds if:

- `e` is a `ThenExecutor` where `make_promise_contract(e)` is well-formed.
- `e` is a `OnewayExecutor` or is convertible to a `OnewayExecutor`.

Fails at compile-time otherwise.

Postconditions: Has no observable affect on `sfh`.

§ 2.7. `std::execution::promise`

```
template <class T>
class promise {
public:
    using value_type = T;

    promise() noexcept;
    promise(promise&&) noexcept;
    template <typename Promise>
    explicit promise(Promise&& p);

    void set_value(/* see below */) &&;

    template <typename Error>
    void set_exception(Error&& err) &&;

    bool valid() const noexcept;
    explicit operator bool() const noexcept;
};
```

A `promise` refers to a `promise` and is associated with a `future`, either through type-erasure or through construction of an underlying `promise` with an overload of `make_promise_contract()`.

```
promise() noexcept;
```

- *Effects:* Constructs a `promise` object that does not refer to a `promise`
- *Postconditions:*
 - `valid() == false`.

```
promise(promise&& rhs) noexcept;
```

- *Effects*: Move constructs a `promise` object from `rhs` that refers to the same `promise` and is associated with the same `future` (if `rhs` refers to a `promise`).
- *Postconditions*:
 - `valid()` returns the same value as `rhs.valid()` prior to the constructor invocation.
 - `rhs.valid() == false`.

```
template <class Promise>
promise(Promise&& rhs) noexcept;
```

- *Requires*:
- `Promise` meets the requirements for `Promise<value_type>`
- `!is_same_v<decay_t<Promise>, promise>`
- `!is_reference_v<Promise>` *Note*: This constrains the parameter to be an rvalue reference rather than a forwarding reference —*end note*
- *Effects*: Constructs a `promise` object that refers to the `promise` `rhs` and is associated with the same `future` as `rhs`. (if `rhs` is associated with a `future`).
- *Postconditions*:
 - `valid()` returns the same value as `rhs.valid()` prior to the constructor invocation.

```
void promise::set_value(const T& val) &&;
void promise::set_value(T&& val) &&;
void promise<void>::set_value() &&;
```

- *Effects*:
- If `valid() == true`,
 - For `promise::set_value(const T& val) &&`: equivalent to calling `set_value(val)` on the `promise` that `*this` refers to.
 - For `promise::set_value(T&& val) &&`: equivalent to calling `set_value(std::move(val))` on the `promise` that `*this` refers to.
 - For `promise<void>::set_value() &&`: equivalent to calling `set_value()` on the `promise` that `*this` refers to.
- Otherwise, throws
- *Throws*: `future_error` with error condition `no_state` if `valid() == false`
- *Postconditions*:
- `valid() == false`
- *Notes*: `promise::set_value(const T& val) &&` does not participate in overload resolution unless `is_copy_constructible_v<decay_t<T>>`

```
template <typename Error>
void set_exception(Error&& err) &&;
```

- *Requires:*
- `Error` is contextually convertible to `exception_ptr`
- *Effects:*
- If `valid() == true`, equivalent to calling `set_exception(exception_ptr{forward<Error>(err)})` on the promise that `*this` refers to.
- Otherwise, throws
- *Throws:* `future_error` with error condition `no_state` if `valid() == false`
- *Postconditions:*
- `valid() == false`

```
bool valid() const noexcept;
explicit operator bool() const noexcept;
```

- *Returns:* `true` if `*this` refers to a promise and the referenced promise is `valid()`, or `false` otherwise

§ 2.8. `std::execution::semi_future`

```
template<class T>
class semi_future {
public:
    using value_type = T;

    semi_future(semi_future&&) = default;
    semi_future(const semi_future&) = delete;

    semi_future& operator=(const semi_future&) = delete;
    semi_future& operator=(semi_future&&) = default;

    template<class E>
    explicit semi_future(continuable_future<T, E>&&);

    template<class E>
    explicit semi_future(shared_future<T, E>&&);

    template<class EI>
    continuable_future<T, EI> via(EI) &&;
};
```

```
continuable_future(semi_future&& rhs);
```

- *Effects:* Move constructs a future object from `rhs` that refers to the same [future](#) and its associated with the same [promise](#) (if `rhs` refers to a [future](#)).
- *Postconditions:*
 - `valid()` returns the same value as `rhs.valid()` prior to the constructor invocation.
 - `rhs.valid() == false`.

```
semi_future(const semi_future& rhs);
```

- *Effects:* Copy constructs a future object from `rhs` that refers to the same [future](#) and its associated with the same [promise](#) (if `rhs` refers to a [future](#)).
- *Postconditions:* `valid()` returns the same value as `rhs.valid()` prior to the constructor invocation. The validity of `rhs` does not change.

```
template<class E>  
semi_future(continuable_future<T, E>&& rhs);
```

- *Effects:* Move constructs a future object from `rhs` that refers to the same [future](#) and its associated with the same [promise](#) (if `rhs` refers to a [future](#)).
- *Postconditions:*
 - `valid()` returns the same value as `rhs.valid()` prior to the constructor invocation.
 - `rhs.valid() == false`.

```
template<class E>  
explicit semi_future(shared_future<T, E>&& rhs);  
  
template<class E>  
explicit semi_future(const shared_future<T, E>& rhs);
```

- *Effects:* Move constructs a future object from `rhs` that refers to the same [future](#) and its associated with the same [promise](#) (if `rhs` refers to a [future](#)).
- *Postconditions:*
 - `valid()` returns the same value as `rhs.valid()` prior to the constructor invocation.
 - `rhs.valid() == false`.

```
continuable_future<T, EI> via(EI ex) &&;
```

- *Effects*: Returns a new future that will complete when this completes but on which continuations will be enqueued to a new executor.
- *Returns*: A `continuable_future` modified to carry executor `ex` of type `EI`.
- *Requires*:
 - `e` is a `ThenExecutor` where `make_promise_contract(e)` is well-formed.
 - `e` is a `OnewayExecutor` or is convertible to a `OnewayExecutor`.
- *Postconditions*: `valid() == false`.

```
bool valid() const noexcept;
```

- *Returns*: Returns true if this is a valid future. False otherwise.

§ 2.9. `std::execution::continuable_future`

```
template<class T, class E>
class continuable_future {
public:
    using value_type = T;
    using executor_type = E;
    using semi_future_type = semi_future<T>;

    continuable_future(const continuable_future&) = delete;
    continuable_future(continuable_future&&) = default;

    continuable_future& operator=(const continuable_future&) = delete;
    continuable_future& operator=(continuable_future&&) = default;

    template<class E>
    explicit continuable_future(shared_future<T, E>&&);
    template<class E>
    explicit continuable_future(const shared_future<T, E>&);

    template<class ReturnFuture, class F>
    ReturnFuture then(FutureContinuation&&) &&;

    template<class EI>
    continuable_future<T, EI> via(EI) &&;

    E get_executor() const;
    semi_future<T> semi() &&;
    shared_future<T, E> share() &&;

    bool valid() const
}
};
```

```
continuable_future(continuable_future&& rhs);
```

- *Effects*: Move constructs a future object from `rhs` that refers to the same [future](#) and its associated with the same [promise](#) (if `rhs` refers to a [future](#)).
- *Postconditions*:
 - `valid()` returns the same value as `rhs.valid()` prior to the constructor invocation.
 - `rhs.valid() == false`.

```
continuable_future(const continuable_future& rhs);
```


- *Effects*: Copy constructs a future object from `rhs` that refers to the same [future](#) and its associated with the same [promise](#) (if `rhs` refers to a [future](#)).
- *Postconditions*: `valid()` returns the same value as `rhs.valid()` prior to the constructor invocation. The validity of `rhs` does not change.

```
template<class E>
explicit continuable_future(shared_future<T, E>&& rhs);
```

- *Effects*: Move constructs a future object from `rhs` that refers to the same [future](#) and its associated with the same [promise](#) (if `rhs` refers to a [future](#)).
- *Postconditions*:
 - `valid()` returns the same value as `rhs.valid()` prior to the constructor invocation.
 - `rhs.valid() == false`.

```
template<class E>
explicit continuable_future(const shared_future<T, E>& rhs);
```

- *Effects*: Copy constructs a future object from `rhs` that refers to the same [future](#) and its associated with the same [promise](#) (if `rhs` refers to a [future](#)).
- *Postconditions*: `valid()` returns the same value as `rhs.valid()` prior to the constructor invocation. The validity of `rhs` does not change.

```
template<class ReturnFuture, class F>
ReturnFuture then(F&&) &&;
```

- *Requires*: `F` satisfies the requirements of `FutureContinuation`.
- *Returns*: A `ContinuableFuture` that is bound to the executor `e` and that wraps the type returned by execution of either the value or exception operations implemented in the continuation. The type of the returned `ContinuableFuture` is defined by `Executor E`.
- *Effects*:
 - For `NORMAL` defined as the expression `DECAY_COPY(std::forward<F>(g)) (val)` if `T` is non-void and `DECAY_COPY(std::forward<F>(g)) ()` if `T` is void.
 - For `EXCEPTIONAL` defined as the expression `DECAY_COPY(std::forward<F>(f)) (exception_arg, ex)`,
 - When `*this` becomes nonexceptionally ready, and if `NORMAL` is a well-formed expression, creates an execution agent which invokes `NORMAL` at most once, with the call to `DECAY_COPY` being evaluated in the thread that called `.then`.
 - Otherwise, when `*this` becomes exceptionally ready, if `EXCEPTIONAL` is a well-formed expression, creates an execution agent which invokes `EXCEPTIONAL` at most once, with the call to `DECAY_COPY` being evaluated in the

thread that called `.then`.

- If `NORMAL` and `EXCEPTIONAL` are both well-formed expressions, `decltype(EXCEPTIONAL)` shall be convertible to `R`.
 - If `NORMAL` is not a well-formed expression and `EXCEPTIONAL` is a well-formed expression, `decltype(EXCEPTIONAL)` shall be convertible to `decltype(val)`.
 - If neither `NORMAL` nor `EXCEPTIONAL` are well-formed expressions, the invocation of `.then` shall be ill-formed.
 - May block pending completion of `NORMAL` or `EXCEPTIONAL`.
 - The invocation of `.then` synchronizes with (C++Std [intro.multithread]) the invocation of `f`.
 - Stores the result of either the `NORMAL` or `EXCEPTIONAL` expression, or any exception thrown by either, in the associated shared state of the resulting `ContinuableFuture`. Otherwise, stores either `val` or `e` in the associated shared state of the resulting `ContinuableFuture`.
- *Postconditions*: `valid() == false`.

```
continuable_future<T, EI> via(EI ex) &&;
```

- *Effects*: Returns a new future that will complete when this completes but on which continuations will be enqueued to a new executor.
- *Returns*: A `continuable_future` modified to carry executor `ex` of type `EI`.
- *Requires*:
- `e` is a `ThenExecutor` where `make_promise_contract(e)` is well-formed.
- `e` is a `OnewayExecutor` or is convertible to a `OnewayExecutor`.
- *Postconditions*: `valid() == false`.

```
E get_executor() const;
```

- *Returns*: If `is_valid()` returns true returns the executor contained within the future. Otherwise throws `std::future_error`.

```
semi_future<T> semi() &&;
```

- *Returns*: Returns a `semi_future` of the same value type as `*this` and that completes when this completes, but that erases the executor. `is_valid()` on the returned `semi_future` will return the same value as `is_valid()` on `*this`.

```
bool valid() const noexcept;
```

- *Returns*: Returns true if this is a valid future. False otherwise.

```
shared_future<T, E> share() &&;
```

- *Returns:* `shared_future (std::move(*this))`.
- *Postconditions:* `valid() == false`.

§ 2.10. `std::execution::shared_future`

```
namespace std::execution {
    template<class T, class E>
    class shared_future {
    public:
        using value_type = T;
        using executor_type = Ex;
        using semi_future_type = semi_future<T>;

        shared_future(const shared_future&) = default;
        shared_future(shared_future&&) = default;

        shared_future& operator=(const shared_future&) = default;
        shared_future& operator=(shared_future&&) = default;

        template<class E>
        explicit shared_future(continuable_future<T, E>&&);

        template<class ReturnFuture, class F>
        ReturnFuture then(FutureContinuation&&);

        template<class EI>
        shared_future<T, EI> via(EI);

        E get_executor() const;
        semi_future<T> semi();

        bool valid() const;
    }
};
```

```
shared_future(shared_future&& rhs);
```

- *Effects:* Move constructs a future object that refers to the shared state that was originally referred to by rhs (if any).
- *Postconditions:*
 - `valid()` returns the same value as `rhs.valid()` prior to the constructor invocation.

- `rhs.valid() == false`.

```
shared_future(const shared_future& rhs);
```

- *Effects*: Copy constructs a future object that refers to the shared state that was originally referred to by `rhs` (if any).
- *Postconditions*: `valid()` returns the same value as `rhs.valid()` prior to the constructor invocation. The validity of `rhs` does not change.

```
explicit shared_future(continuable_future&& rhs);
```

- *Effects*: Move constructs a future object that refers to the shared state that was originally referred to by `rhs` (if any).
- *Postconditions*:
 - `valid()` returns the same value as `rhs.valid()` prior to the constructor invocation.
 - `rhs.valid() == false`.

```
template<class ReturnFuture, class F>
ReturnFuture then(F&&);
```

- *Requires*: `F` satisfies the requirements of `FutureContinuation`.
- *Returns*: A `ContinuableFuture` that is bound to the executor `e` and that wraps the type returned by execution of either the value or exception operations implemented in the continuation. The type of the returned `ContinuableFuture` is defined by `Executor E`.
- *Effects*:
 - For `NORMAL` defined as the expression `DECAY_COPY(std::forward<F>(g))(val)` if `T` is non-void and `DECAY_COPY(std::forward<F>(g))()` if `T` is void.
 - For `EXCEPTIONAL` defined as the expression `DECAY_COPY(std::forward<F>(f))(exception_arg, ex)`,
 - When `*this` becomes nonexceptionally ready, and if `NORMAL` is a well-formed expression, creates an execution agent which invokes `NORMAL` at most once, with the call to `DECAY_COPY` being evaluated in the thread that called `.then`.
 - Otherwise, when `*this` becomes exceptionally ready, if `EXCEPTIONAL` is a well-formed expression, creates an execution agent which invokes `EXCEPTIONAL` at most once, with the call to `DECAY_COPY` being evaluated in the thread that called `.then`.
 - If `NORMAL` and `EXCEPTIONAL` are both well-formed expressions, `decltype(EXCEPTIONAL)` shall be convertible to `R`.
 - If `NORMAL` is not a well-formed expression and `EXCEPTIONAL` is a well-formed expression, `decltype(EXCEPTIONAL)` shall be convertible to `decltype(val)`.
 - If neither `NORMAL` nor `EXCEPTIONAL` are well-formed expressions, the invocation of `.then` shall be ill-formed.

- May block pending completion of `NORMAL` or `EXCEPTIONAL`.
- The invocation of `.then synchronizes with` (C++Std [intro.multithread]) the invocation of `f`.
- Stores the result of either the `NORMAL` or `EXCEPTIONAL` expression, or any exception thrown by either, in the associated shared state of the resulting `ContinuableFuture`. Otherwise, stores either `val` or `e` in the associated shared state of the resulting `ContinuableFuture`.
- *Postconditions*: No observable change to `*this`.

```
shared_future<T, EI> via(EI ex);
```

- *Effects*: Returns a new future that will complete when this completes but on which continuations will be enqueued to a new executor.
- *Returns*: A `shared_future` modified to carry executor `ex` of type `EI`.
- *Requires*:
- `e` is a `ThenExecutor` where `make_promise_contract(e)` is well-formed.
- `e` is a `OnewayExecutor` or is convertible to a `OnewayExecutor`.
- *Postconditions*: No observable change to `*this`.

```
E get_executor() const;
```

- *Returns*: If `is_valid()` returns true returns the executor contained within the future. Otherwise throws `std::future_error`.

```
semi_future<T> semi();
```

- *Returns*: Returns a `semi_future` of the same value type as `*this` and that completes when this completes, but that erases the executor. `is_valid()` on the returned `semi_future` will return the same value as `is_valid()` on `*this`.

```
bool valid() const noexcept;
```

- *Returns*: Returns true if this is a valid future. False otherwise.

§ 2.11. `std::execution::make_promise_contract`

```
template <class T, class Executor>
/* see below */
make_promise_contract(const Executor& ex)
    requires execution::is_then_executor_v<Executor> && execution::can_query_v<Executor, prom
```

- *Effects:* equivalent to `execution::query(ex, promise_contract_t<T>{})`
- *Returns:* same as `execution::query(ex, promise_contract_t<T>{})`

```
template <class T, class Executor>
pair<promise<T>, continuable_future<T, decay_t<Executor>>>
make_promise_contract(const Executor& ex)
    requires execution::is_one_way_executor_v<Executor>
```

- *Returns:* A pair of:
 - a `promise` that refers to a `promise` that is associated with the `future` in the pair
 - a `continuable_future` that is associated with the `promise` and is bound to `ex`.

```
template <class T>
pair<promise<T>, semi_future<T>>
make_promise_contract()
```

- *Returns:* A pair of:
 - a `promise` that refers to a `promise` that is associated with the `future` in the pair
 - a `semi_future` that is associated with the `promise`.

§ 2.12. Generic Future Blocking Functions

In `[thread.syn]` and `[thread.thread.this]` add:

```
namespace std::this_thread {

    template<class Future>
        void future_wait(Future& f) noexcept;

    template<class Future>
        future_value_t<decay_t<Future>> future_get(Future&& f);

}
```

In `[thread.thread.this]` add:

```
template<class Future>
    void future_wait(Future& f) noexcept;
```

- *Requires:* `Future` meets the requirements of `SemiFuture`.
- *Effects:* Blocks the calling thread until `f` becomes ready.
- *Synchronization:* The destruction of the continuation that fulfills `f`'s synchronizes with `future_wait` calls blocking until `f` becomes ready.

```
template<class Future>
    future_value_t<decay_t<Future>> future_get(Future&& f);
```

- *Requires:*
- `decay_t<Future>` shall meet the `SemiFuture` requirements.
- `!is_reference_v<Future>` [*Note:* This constrains the parameter to be an rvalue reference rather than a forwarding reference. — *end note*]
- *Effects:*
- Blocks the calling thread until `f` becomes ready.
- Retrieves the asynchronous result.
- *Returns:*
 - If `f` becomes ready with a value, moves the value from `f` and returns it to the caller.
- *Postconditions:* `f` is invalid.
- *Synchronization:* The destruction of the continuation that generates `f`'s value synchronizes with `future_get` calls blocking until `f` becomes ready.
- *Throws:* If `f` becomes ready with an exception, that exception is rethrown.

§ 2.13. `FutureContinuation` Helper Functions

```
namespace std::execution {

template <class F>
    /* see below */ on_value(F&& f);

template<class F>
    /* see below */ on_error(F&& f);

template<class F, class G>
    /* see below */ on_value_or_error(F&& f, G&& g);

}
```

```
template<class F>
/* see below */ on_value(F&& f);
```

Requires:

- For any (possibly cv-qualified) object type `T`, `invoke(declval<F>(), declval<T>())` shall be well-formed.
- `F` shall meet the `MoveConstructible` requirements.

Returns: A `FutureContinuation` object, `fc`, of implementation-defined type such that:

- `fc(t)` is well-formed for objects `t` of type `T` and has the same effects as `invoke(ff, t)`, where `ff` is an instance of `F` move-constructed from `forward<F>(f)`. The object `ff` is constructed before the return of `on_value()` and destroyed when `fc` is destroyed.
- `fc(exception_arg, exception_ptr{})` is ill-formed.

```
template<class F>
/* see below */ on_error(F&& f);
```

Requires:

- `invoke(declval<F>(), declval<exception_ptr>())` shall be well-formed.
- `F` shall meet the `MoveConstructible` requirements.

Returns: A `FutureContinuation` object, `fc`, of implementation-defined type such that:

- `fc(exception_tag, e)` is well-formed for objects `e` of type `exception_ptr` and has the same effects as `invoke(ff, e)`, where `ff` is an object of type `F` move-constructed from `forward<F>(f)`. The object `ff` is constructed before the return of `on_error` and destroyed when `fc` is destroyed.
- `fc(t)` is ill-formed for any type `T`.

```
template<class F, class G>
/* see below */ on_value_or_error(F&& f, G&& g);
```

Requires:

- For any (possibly cv-qualified) object type `T`, `invoke(declval<F>(), declval<T>())` and `invoke(declval<G>(), declval<exception_ptr>())` shall be well-formed.
- `F` and `G` shall meet the `MoveConstructible` requirements.

Returns: A `FutureContinuation` object, `fc`, of implementation-defined type such that:

- `fc(t)` is well-formed for objects `t` of type `T` and has the same effects as `invoke(ff, t)`, where `ff` is an object of type `F` move-constructed from `forward<F>(f)`. The object `ff` is constructed before the return of `on_value_or_error` and destroyed when `fc` is destroyed.
- `fc(exception_tag, e)` is well-formed for objects `e` of type `exception_ptr` and has the same effects as `invoke(gg, e)`, where `gg` is an object of type of `G` move-constructed from `forward<G>(g)`. The object `gg` is constructed before the return of `on_value_or_error` and destroyed when `fc` is destroyed.

§ 2.14. Proposed Modifications to Executors

§ 2.14.1. `Future` Requirements

Remove this section.

§ 2.14.2. `TwoWayExecutor` Requirements

In the *Return Type* column:

Replace:

A type that satisfies the `Future` requirements for the value type `R`.

With:

A type that satisfies the `ContinuableFuture` requirements for the value type `R`.

In the *Operational Semantics* column:

Replace:

in the associated shared state of the resulting `Future`.

With:

in the resulting `ContinuableFuture`.

§ 2.14.3. `ThenExecutor` Requirements

In the type requirements list:

Replace:

- `fut` denotes a future object satisfying the `Future` requirements,

With:

- `fut` denotes a future object that:
- was returned by a call to `x.twoway_execute`, `x.bulk_twoway_execute`, `x.then_execute`, or `x.bulk_then_execute` and meets the `ContinuableFuture` requirements.
- was returned by a call to `execution::query(x, promise_contract_t<T>)` or `execution::query(x, cancellable_promise_contract_t<T>)`.

In the *Return Type* column:

Replace:

A type that satisfies the `Future` requirements for the value type `R`.

With:

A type that satisfies the `ContinuableFuture` requirements for the value type `R`.

In the *Operational Semantics* column:

Replace:

in the associated shared state of the resulting `Future`.

With:

in the resulting `ContinuableFuture`.

§ 2.14.4. `BulkTwoWayExecutor` Requirements

In the *Return Type* column:

Replace:

A type that satisfies the `Future` requirements for the value type `R`.

With:

A type that satisfies the `ContinuableFuture` requirements for the value type `R`.

In the *Operational Semantics* column:

Replace:

in the associated shared state of the resulting `Future`.

With:

in the resulting `ContinuableFuture`.

§ 2.14.5. `BulkThenExecutor` Requirements

In the type requirements list:

Replace:

- `fut` denotes a future object satisfying the `Future` requirements,

With:

- `fut` denotes a future object that:
- was returned by a call to `x.twoway_execute`, `x.bulk_twoway_execute`, `x.then_execute`, or `x.bulk_then_execute` and meets the `ContinuableFuture` requirements.
- was returned by a call to `execution::query(x, promise_contract_t<T>)` or `execution::query(x, cancellable_promise_contract_t<T>)`.

In the *Return Type* column:

Replace:

A type that satisfies the `Future` requirements for the value type `R`.

With:

A type that satisfies the `ContinuableFuture` requirements for the value type `R`

In the *Operational Semantics* column:

Replace:

in the associated shared state of the resulting `Future`.

With:

in the resulting `ContinuableFuture`.

§ 2.14.6. `twoway_t` Customization Points

Replace:

it is `std::experimental::future<T>`

With:

it is a `execution::continuable_future<T, E1>`

§ 2.14.7. `single_t` Customization Points

Replace:

it is `std::experimental::future<T>`

With:

it is `execution::continuable_future<T, E1>`

§ 2.14.8. Properties To Indicate If Blocking And Directionality May Be Adapted

Remove `twoway_t` from the **Requirements** column of the Table.

§ 2.14.9. Class Template `executor`

Replace:

```
template<class Function>
    std::experimental::future<result_of_t<decay_t<Function>()>>
        twoway_execute(Function&& f) const
```

With:

```
template<class Function>
    execution::semi_future<result_of_t<decay_t<Function>()>>
        twoway_execute(Function&& f) const
```

Replace:

```
template<class Function, class ResultFactory, class SharedFactory>
    std::experimental::future<result_of_t<decay_t<ResultFactory>()>>
        bulk_twoway_execute(Function&& f, size_t n, ResultFactory&& rf, SharedFactory&& sf)
```

With:

```
template<class Function, class ResultFactory, class SharedFactory>
    execution::semi_future<result_of_t<decay_t<ResultFactory>()>>
        bulk_twoway_execute(Function&& f, size_t n, ResultFactory&& rf, SharedFactory&& sf)
```

§ 2.14.10. `executor` Operations

Replace:

```
template<class Function>
std::experimental::future<result_of_t<decay_t<Function>()>>
    twoway_execute(Function&& f) const
```

With:

```
template<class Function>
/* implementation-defined future type */
    twoway_execute(Function&& f) const
```

Replace:

Returns: A future, whose shared state is made ready when the future returned by `e.twoway_execute(f2)` is made ready, containing the result of `f1()` or any exception thrown by `f1()`. [*Note:* `e2.twoway_execute(f2)` may return any future type that satisfies the `Future` requirements, and not necessarily One possible implementation approach is for the polymorphic wrapper to attach a continuation to the inner future via that object's `then()` member function. When invoked, this continuation stores the result in the outer future's associated shared state and makes that shared state ready. — *end note*]

With:

Returns: A value whose type satisfies the `ContinuableFuture` requirements. The returned future is `fulfilled` when `f1()` completes execution, with the result of `f1()` (if `decltype(f1())` is `void`), valueless completion (if `decltype(f1())` is `void`), or any exception thrown by `f1()`.

Replace:

```
template<class Function, class ResultFactory, class SharedFactory>
std::experimental::future<result_of_t<decay_t<ResultFactory>()>>
    void bulk_twoway_execute(Function&& f, size_t n, ResultFactory&& rf, SharedFactory&&
```

With:

```
template<class Function, class ResultFactory, class SharedFactory>
/* implementation-defined future type */
    bulk_twoway_execute(Function&& f, size_t n, ResultFactory&& rf, SharedFactory&& sf)
```

Replace:

Returns: A future, whose shared state is made ready when the future returned by `e.bulk_twoway_execute(f2, n, rf2, sf2)` is made ready, containing the result in `r1` (if `decltype(rf1())` is non-void) or any exception thrown by an invocation `f1`. [*Note:* `e.bulk_twoway_execute(f2)` may return any future type that satisfies the `Future` requirements, and not necessarily `std::experimental::future`. One possible implementation approach is for the polymorphic wrapper to attach a continuation to the inner future via that object's `then()` member function. When invoked, this continuation stores the result in the outer future's associated shared state and makes that shared state ready. — *end note*]

With:

Returns:

A value whose type satisfies the `ContinuableFuture` requirements. The returned future is `fulfilled` when `f1()` completes execution, with the result in `r1` (if `decltype(rf1())` is `void`), valueless completion (if `decltype(rf1())` is `void`), or any exception thrown by an invocation `f1`.

§ 2.14.11. `static_thread_pool` Executor Type

Replace:

```
template<class Function>
std::experimental::future<result_of_t<decay_t<Function>()>>
twoway_execute(Function&& f) const;

template<class Function, class Future>
std::experimental::future<result_of_t<decay_t<Function>(decay_t<Future>)>>
then_execute(Function&& f, Future&& pred) const;

template<class Function, class SharedFactory>
void bulk_execute(Function&& f, size_t n, SharedFactory&& sf) const;

template<class Function, class ResultFactory, class SharedFactory>
std::experimental::future<result_of_t<decay_t<ResultFactory>()>>
void bulk_twoway_execute(Function&& f, size_t n, ResultFactory&& rf, SharedFactory&&
```

With:

```

template<class Function>
    std::execution::continuable_future<result_of_t<decay_t<Function>()>, C>
        twoway_execute(Function&& f) const;

template<class Function, class Future>
    execution::continuable_future<result_of_t<decay_t<Function>(decay_t<Future>)>, C>
        then_execute(Function&& f, Future&& pred) const;

template<class Function, class SharedFactory>
    void bulk_execute(Function&& f, size_t n, SharedFactory&& sf) const;

template<class Function, class ResultFactory, class SharedFactory>
    execution::continuable_future<result_of_t<decay_t<ResultFactory>()>>, C>
        void bulk_twoway_execute(Function&& f, size_t n, ResultFactory&& rf, SharedFactory&&

```

Replace the same instances in the documentation section below the main code block.

§ Index

§ Terms defined by this specification

<u>asynchronous result</u> , in §2.1	<u>cancels</u> , in §2.1	<u>future value propagation</u> , in §1.3
<u>attach</u> , in §2.1	<u>continuable future</u> , in §2.5	<u>invalid</u> , in §2.1
<u>attached</u> , in §2.1	<u>continuations</u> , in §2.1	<u>non-uniquely owned future</u> , in §2.1
<u>attaches</u> , in §2.1	<u>fulfill</u> , in §2.1	<u>not ready</u> , in §2.1
<u>attachment</u> , in §2.1	<u>fulfilled</u> , in §2.1	<u>promise</u> , in §2.1
<u>cancel</u> , in §2.1	<u>fulfillment</u> , in §2.1	<u>ready</u> , in §2.1
<u>cancellation</u> , in §2.1	<u>fulfills</u> , in §2.1	<u>semi future</u> , in §2.4
<u>cancellation notifier</u> , in §2.1	<u>future</u> , in §2.1	<u>shared future</u> , in §2.6
<u>cancelled</u> , in §2.1	<u>future continuation</u> , in §2.2	<u>status</u> , in §2.1
	<u>future exception propagation</u> , in §1.3	<u>uniquely owned future</u> , in §2.1

§ References

§ Informative References

[P0701r0]

Bryce Adelstein Lelbach. [Back to the std2::future](#). URL: <https://wg21.link/p0701r0>

[P0701r1]

Bryce Adelstein Lelbach. [Back to the std2::future](#). URL: <https://wg21.link/p0701r1>

[P0783r0]

Lee Howes, Andrii Grynenko, Jay Feldblum. [Continuations without overcomplicating the future](https://wg21.link/p0783r0). URL: <https://wg21.link/p0783r0>

[P0904r0]

Lee Howes, Andrii Grynenko, Jay Feldblum. [A strawman Future API](https://wg21.link/p0904r0). URL: <https://wg21.link/p0904r0>